

Manual técnico — Proyecto 1

1. Información general

Curso: Laboratorio de Sistemas Operativos 1

Estudiante: Victor Hugo Velásquez Hernández

Carné: 202100054

Repositorio: https://github.com/vjr-velasquez/202100054_LAB_SO1_1S2026

2. Descripción del proyecto

El proyecto consiste en implementar tres APIs desarrolladas en Go y distribuir las en diferentes máquinas virtuales.

Cada máquina utiliza una tecnología distinta para administrar los contenedores:

- VM1 utiliza containerd para ejecutar API1 y API2.
- VM2 utiliza Podman para ejecutar API3.
- VM3 utiliza Docker para ejecutar el registro privado Zot.

Las imágenes de las tres APIs se almacenan en Zot. Posteriormente, VM1 y VM2 descargan las imágenes desde este registro y ejecutan los contenedores correspondientes.

3. Objetivos

Objetivo general

Implementar una arquitectura distribuida utilizando diferentes motores de contenedores y un registro privado de imágenes OCI.

Objetivos específicos

- Desarrollar tres APIs capaces de comunicarse mediante HTTP.
- Ejecutar API1 y API2 con containerd.
- Ejecutar API3 con Podman.
- Ejecutar Zot mediante Docker.
- Publicar y descargar imágenes desde un registro privado.

- Comprobar la comunicación entre las APIs.

4. Arquitectura

Máquina	Dirección IP	Tecnología	Servicio
VM1	192.168.122.76	containerd, nerdctl, BuildKit y CNI	API1 y API2
VM2	192.168.122.252	Podman	API3
VM3	192.168.122.181	Docker y Zot	Registro de imágenes

Puertos utilizados

Servicio	Puerto
API1	8081
API2	8082
API3	8083
Zot	5000

Flujo de imágenes

```
API1 y API2 → nerdctl/BuildKit → Zot → containerd
API3          → Podman              → Zot → Podman
```

Comunicación entre APIs

```
API1 → API2
API1 → API3
API2 → API3
API3 → API1
API3 → API2
```

Evidencia sugerida: insertar una captura o diagrama de las tres máquinas virtuales.

5. Tecnologías utilizadas

Tecnología	Versión	Uso
Go	Imagen <code>golang:1.24-alpine</code>	Desarrollo y compilación de las APIs
containerd	2.2.1	Ejecución de API1 y API2
nerdctl	2.3.1	Administración de containerd
BuildKit	0.32.2	Construcción de imágenes en VM1
CNI plugins	1.9.1	Redes de contenedores en VM1
Podman	3.4.4	Construcción y ejecución de API3
Docker	29.1.3	Ejecución de Zot
Zot	Imagen oficial	Registro privado OCI

6. Estructura del repositorio

```

202100054_LAB_S01_1S2026/
├── api1/
│   ├── Dockerfile
│   ├── .dockerignore
│   └── main.go
├── api2/
│   ├── Dockerfile
│   ├── .dockerignore
│   └── main.go
├── api3/
│   ├── Dockerfile
│   ├── .dockerignore
│   └── main.go
├── .gitignore
└── README.md

```

Cada Dockerfile utiliza una construcción multietapa. La primera etapa compila el programa con Go y la segunda utiliza `scratch` para generar una imagen final pequeña.

Las imágenes finales de las APIs tienen un tamaño aproximado de 6.3 MB.

7. Construcción de las imágenes

7.1 API1 y API2 en VM1

API1 fue construida con:

```
cd /home/vjr/proyecto1/api1
sudo nerdctl build --progress=plain -t proyecto1-api1:v1 .
```

API2 fue construida con:

```
cd /home/vjr/proyecto1/api2
sudo nerdctl build --progress=plain -t proyecto1-api2:v1 .
```

Las imágenes se verificaron con:

```
sudo nerdctl images
```

| Evidencia sugerida: captura de `sudo nerdctl images` mostrando API1 y API2.

7.2 API3 en VM2

API3 fue construida mediante Podman:

```
cd /home/vjr/proyecto1/api3
podman build --format docker -t proyecto1-api3:v1 .
```

La imagen se verificó con:

```
podman images
```

| Evidencia sugerida: captura de `podman images` mostrando API3.

8. Configuración del registro Zot

Zot se ejecuta en VM3 mediante Docker y utiliza un volumen para conservar las imágenes.

Creación del volumen

```
sudo docker volume create zot-data
```

Ejecución del registro

```
sudo docker run -d \
  --name zot \
  --restart=unless-stopped \
  -p 5000:5000 \
  -v zot-data:/var/lib/registry:z \
  ghcr.io/project-zot/zot-linux-amd64:latest
```

Verificación

```
sudo docker ps
```

```
curl -s -o /dev/null \
  -w 'HTTP %{http_code}\n' \
  http://localhost:5000/v2/
```

Zot respondió con el código:

```
HTTP 200
```

Evidencia sugerida: captura de `sudo docker ps` mostrando el contenedor Zot.

9. Publicación de las imágenes

API1 y API2 desde VM1

```
sudo nerdctl tag proyecto1-api1:v1 \
  192.168.122.181:5000/proyecto1-api1:v1
```

```
sudo nerdctl tag proyecto1-api2:v1 \
  192.168.122.181:5000/proyecto1-api2:v1
```

```
sudo nerdctl --insecure-registry push \
  192.168.122.181:5000/proyecto1-api1:v1
```

```
sudo nerdctl --insecure-registry push \
  192.168.122.181:5000/proyecto1-api2:v1
```

API3 desde VM2

```
podman tag localhost/proyecto1-api3:v1 \
  192.168.122.181:5000/proyecto1-api3:v1
```

```
podman push --tls-verify=false \
  192.168.122.181:5000/proyecto1-api3:v1
```

Catálogo del registro

```
curl http://192.168.122.181:5000/v2/_catalog
```

Resultado:

```
{
  "repositories": [
    "proyecto1-api1",
    "proyecto1-api2",
    "proyecto1-api3"
  ]
}
```

Evidencia sugerida: captura del catálogo de Zot con las tres imágenes.

10. Despliegue de las APIs

API1 en VM1

```
sudo nerdctl run -d \
  --name api1-zot \
  --restart=unless-stopped \
  --network proyecto1-net \
  -p 8081:8081 \
```

```
-e API2_URL=http://192.168.122.76:8082 \  
-e API3_URL=http://192.168.122.252:8083 \  
192.168.122.181:5000/proyecto1-api1:v1
```

API2 en VM1

```
sudo nerdctl run -d \  
  --name api2-zot \  
  --restart=unless-stopped \  
  --network proyecto1-net \  
  -p 8082:8082 \  
  -e API1_URL=http://192.168.122.76:8081 \  
  -e API3_URL=http://192.168.122.252:8083 \  
  192.168.122.181:5000/proyecto1-api2:v1
```

API3 en VM2

```
podman run -d \  
  --name api3-zot \  
  --restart=unless-stopped \  
  -p 8083:8083 \  
  -e API1_URL=http://192.168.122.76:8081 \  
  -e API2_URL=http://192.168.122.76:8082 \  
  192.168.122.181:5000/proyecto1-api3:v1
```

11. Pruebas realizadas

Estado de los servicios

```
curl http://192.168.122.76:8081/health  
curl http://192.168.122.76:8082/health  
curl http://192.168.122.252:8083/health
```

Las tres APIs respondieron con estado **UP**.

Comunicación cruzada

```
curl http://192.168.122.76:8081/api1/202100054/call-api2
curl http://192.168.122.76:8081/api1/202100054/call-api3
curl http://192.168.122.76:8082/api2/202100054/call-api3
curl http://192.168.122.252:8083/api3/202100054/call-api1
curl http://192.168.122.252:8083/api3/202100054/call-api2
```

Todas las pruebas devolvieron:

```
{
  "connection": true,
  "carnet": "202100054"
}
```

Evidencia sugerida: captura de las cinco respuestas con `"connection":true`.

12. Conclusiones

Se implementó correctamente una arquitectura distribuida con tres APIs y tres tecnologías de contenedores.

API1 y API2 se ejecutaron con containerd, API3 con Podman y el registro Zot con Docker. Las imágenes fueron publicadas y descargadas desde el registro privado.

Las pruebas confirmaron que las tres APIs permanecen disponibles y pueden comunicarse correctamente entre VM1 y VM2.